

Unit 2: Introduction to AJAX

Introduction to AJAX

AJAX stands for **Asynchronous JavaScript and XML**. It is a web development technique used for creating interactive and fast web applications. It is a technique that enables web pages to send and receive data asynchronously from a web server, without having to reload the entire web page. This makes the web application faster and more responsive to user interactions. When a user performs an action like clicking a button or typing into a search bar, AJAX allows the application to send a small request to the server and update just the necessary part of the page with the response.

For example, when you start typing a search query on Google, you receive real-time suggestions. This happens because AJAX sends your partial input to the server and retrieves matching results without reloading the page.

Another example is in email applications like Gmail, where messages can be loaded or marked as read instantly without refreshing the inbox.

AJAX combines multiple technologies:

- HTML and CSS for content and presentation
- JavaScript for logic and interactivity
- XMLHttpRequest object for data exchange with server
- Data formats like XML or JSON to carry data between client and server

Use Cases of AJAX:

- Google search suggestions as you type
- Live form validation
- Real-time chat updates
- Auto-save features in editors like Google Docs

AJAX Naming Rules

When writing AJAX code using JavaScript, it's important to follow standard naming rules to avoid errors and keep the code clean:

1. Use meaningful variable names, e.g., xhttp for XMLHttpRequest object.
2. Variable and function names should not use spaces.
3. Names must begin with a letter, underscore `_`, or dollar sign `$`, and should not start with numbers.
4. Names are case-sensitive (myVar and myvar are different).

5. Avoid using JavaScript reserved keywords (like var, function, return) as variable or function names.
6. Use camelCase for naming functions and variables (e.g., sendData, checkUser).

Structure of AJAX Code

```
var xhttp = new XMLHttpRequest();  
xhttp.onreadystatechange = function() {  
    if (this.readyState == 4 && this.status == 200) {  
        document.getElementById("output").innerHTML = this.responseText;  
    }  
};  
xhttp.open("GET", "data.txt", true);  
xhttp.send();
```

In the above example structure:

- We create a new XMLHttpRequest object.
- We define what should happen when the state of the request changes.
- We check if the request is completed (readyState == 4) and successful (status == 200).
- If successful, we update a part of the webpage with the server response.
- open() initializes the request.
- send() sends the request to the server.

AJAX Web Application Model vs Traditional Model

In the traditional web model, each user request results in a full-page reload. When you click a button, submit a form, or navigate to another page, the entire page is requested again from the server. This can be slow and inefficient, especially if only a small portion of the content changes.

In the AJAX model, JavaScript running in the browser sends asynchronous requests to the server in the background. The server returns only the required data, which is then dynamically inserted into the current page using JavaScript. This model improves performance, reduces bandwidth usage, and makes the user experience smoother.

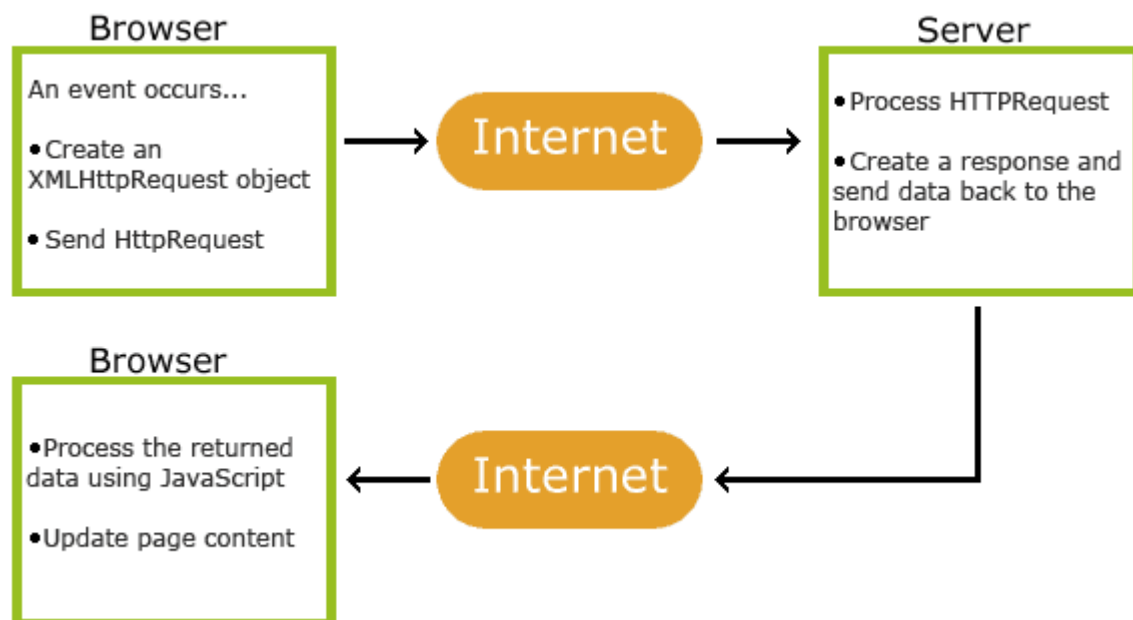
For instance, instead of reloading a product page just to update the price or quantity, AJAX can fetch the updated value and change it on the same page instantly.

Comparison Table:

Feature	Traditional Model	AJAX Model
Page Reload	Full page reload on every action	No full reload; updates happen dynamically
Data Transfer	Entire page data sent	Only required data sent
Performance	Slower and less efficient	Faster and more responsive
User Experience	Interrupts user flow	Smooth and seamless
Bandwidth Usage	Higher (entire page loads each time)	Lower (only partial data sent/received)

Working of AJAX

The core of AJAX is the XMLHttpRequest object (to send and receive data asynchronously), which handles all server communication.



Working:

Step 1: Event Triggered in Browser

- A user action such as clicking a button or typing in a text field triggers an event.
- JavaScript responds by creating an XMLHttpRequest object.
- The request is prepared and sent to the server.

Step 2: Server Processes Request

- The server receives the HTTP request sent from the browser.
- It processes the request using backend logic (e.g., PHP, Node.js).
- A response is created (could be data from a database, a file, or a computed result).
- This response is sent back to the browser via the internet.

Step 3: Browser Receives and Processes Response (Updates Page)

- The browser, using JavaScript, receives the server's response.
- JavaScript parses the response (text, XML, or JSON).
- Only the required portion of the web page is updated using DOM manipulation.

Overall in this Process:

- The full page is never reloaded.
- Only required data is fetched.
- Results in a smooth and fast user experience.

This cycle of request → response → page update is the core of the AJAX model, enabling real-time web interactions.

Creating the XMLHttpRequest Object

To work with AJAX, the very first step is to create an XMLHttpRequest object. This object is built into modern web browsers and is used to send requests to the server and receive responses.

Syntax for Modern Browsers:

```
var xhttp = new XMLHttpRequest();
```

- This code creates a new instance of the XMLHttpRequest object.
- It is supported in all major browsers including Chrome, Firefox, Safari, Edge, and modern versions of Internet Explorer.
- Once created, this object can be used to define request type, target URL, send data, and process the server response.

Syntax for Older Internet Explorer Versions (IE5 and IE6):

```
var xhttp = new ActiveXObject("Microsoft.XMLHTTP");
```

- Older versions of IE did not support the XMLHttpRequest object natively.
- Instead, they required the use of ActiveX technology to simulate the same functionality.

Cross-browser Compatible Code Example:

```
var xhttp;  
  
if (window.XMLHttpRequest) {  
    // Modern browsers  
    xhttp = new XMLHttpRequest();  
} else {  
    // For older IE browsers  
    xhttp = new ActiveXObject("Microsoft.XMLHTTP");  
}
```

This is the way to ensure that the AJAX application runs smoothly across all versions of browsers by handling both modern and older implementations. Creating the XMLHttpRequest object is the foundation for using AJAX effectively. Once we have this object, we can move ahead with configuring the request, sending data, and handling the response dynamically.

XMLHttpRequest Properties and States

The XMLHttpRequest object has several important properties:

- **onreadystatechange:** A function that runs every time the readyState changes.
- **readyState:** An integer that represents the state of the request.
- **status:** The HTTP status code returned by the server (e.g., 200 for success).

readyState Values and Examples:

1. readyState = 0 (UNSENT):

The request has been created but not yet initialized.

Example:

```
var xhttp = new XMLHttpRequest();  
console.log(xhttp.readyState);           // Outputs 0
```

2. readyState = 1 (OPENED):

The open() method has been called.

Example:

```
xhttp.open("GET", "data.txt", true);
```

```
console.log(xhr.readyState);
```

```
// Outputs 1
```

3. **readyState = 2 (HEADERS_RECEIVED):**

The request has been sent, and headers and status are available.

Example:

```
xhr.send();
```

// After sending, readyState becomes 2 (not immediately accessible)

4. **readyState = 3 (LOADING):**

The browser has received part of the response from the server, but the operation is not complete yet.

Useful for displaying loading indicators.

Example:

```
if (xhr.readyState == 3) {  
    console.log("Receiving data from the server...");  
}
```

5. **readyState = 4 (DONE):**

The operation is complete. Full response is available.

Example:

```
xhr.onreadystatechange = function() {  
    if (this.readyState == 4 && this.status == 200) {  
        console.log(this.responseText);    // Full response from server  
    }  
};
```

Common HTTP Status Codes:

- 200: OK – The request was successful.
- 404: Not Found – The requested resource could not be found.
- 500: Internal Server Error – A server-side error occurred.

XMLHttpRequest Methods

Several methods are available in the XMLHttpRequest object:

1. **open(method, url, async)**

Prepares the request with the HTTP method (GET, POST), the target URL, and whether it should be asynchronous (true or false).

Example:

```
xhttp.open("GET", "data.txt", true);
```

2. **send()**

Sends the request to the server. If it's a GET request, parameters are sent in the URL. For POST requests, use send(data).

Example:

```
xhttp.send(); // For GET request with no body
```

```
xhttp.send("name=John&age=25"); // For POST request with data
```

3. **setRequestHeader(header, value)**

Sets the value of an HTTP request header. Must be called after open() but before send().

Typically used in POST requests to indicate the type of data being sent.

Example:

```
xhttp.open("POST", "submit.php", true);
```

```
xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
```

```
xhttp.send("name=John&age=25");
```

4. **abort()**

Cancels an ongoing request.

Example:

```
xhttp.abort();
```

5. **getAllResponseHeaders()**

Returns all the response headers as a string.

Example:

```
console.log(xhttp.getAllResponseHeaders());
```

Output:

Content-Type: application/json

Date: Mon, 11 Aug 2025 10:00:00 GMT

Server: Apache

Content-Length: 1234

6. `getResponseHeader(header)`

Returns the string value of a specific response header.

Example:

```
console.log(xhttp.getResponseHeader("Content-Type"));
```

Output:

text/plain

Synchronous vs Asynchronous Requests

AJAX requests can be made either synchronously or asynchronously.

Synchronous Requests:

In a synchronous request, JavaScript will stop executing the rest of the code until the request has completed and a response has been received from the server. This means the browser is essentially blocked during this time and the user cannot interact with the page until the request is complete.

This behavior can freeze the browser, especially if the server is slow to respond or the user has a slow network connection.

Example:

```
var xhttp = new XMLHttpRequest();  
xhttp.open("GET", "data.txt", false); // false makes the request synchronous  
xhttp.send();  
document.getElementById("output").innerHTML = xhttp.responseText;
```

In the above example:

- The call to `xhttp.send()` blocks the entire execution.
- The next line (DOM update) will not execute until the response is fully received.

- The user cannot scroll, click, or type on the page until the data is loaded.

Asynchronous Requests:

In asynchronous requests, JavaScript can continue executing the code after sending the request to the server. The browser stays responsive, and once the server sends back the response, a callback function handles it.

This is the recommended and default mode of AJAX, as it ensures that the UI remains interactive while data is being fetched.

Example:

```
var xhttp = new XMLHttpRequest();  
xhttp.onreadystatechange = function() {  
    if (xhttp.readyState == 4 && xhttp.status == 200) {  
        document.getElementById("output").innerHTML = xhttp.responseText;  
    }  
};  
xhttp.open("GET", "data.txt", true); // true makes the request asynchronous  
xhttp.send();
```

In the above example:

- The request is sent, and the browser continues executing other code.
- When the response is ready (readyState == 4), the content is updated.
- The page doesn't freeze, and the user can still interact with the page during the process.

Performing Tasks Using XMLHttpRequest

The XMLHttpRequest object allows to communicate with a server from the background using JavaScript. This helps in creating modern, dynamic web applications that can update parts of a page without requiring a full page reload. Below are the common tasks that can be performed using XMLHttpRequest:

The XMLHttpRequest object enables the following tasks:

1. Loading content dynamically, such as comments, chat messages, or product listings.
2. Submitting form data without refreshing the page.
3. Displaying live updates, like scores or notifications.
4. Communicating with the server using either GET or POST.

5. Receiving and processing data in XML, JSON, or plain text formats.

1. Loading Content Dynamically

This means loading content like comments, reviews, chat messages, or product listings **after** the page is already loaded. The content is fetched from the server and inserted into a specific section of the page.

Example:

```
var xhttp = new XMLHttpRequest();
xhttp.onreadystatechange = function() {
    if (this.readyState == 4 && this.status == 200) {
        document.getElementById("commentsSection").innerHTML = this.responseText;
    }
};
xhttp.open("GET", "get-comments.php", true);
xhttp.send();
```

This fetches updated comments from a server-side file (get-comments.php) and displays them inside a div with ID commentsSection.

2. Submitting Form Data Without Refreshing the Page

AJAX allows to send form data (like name, email, feedback) to the server using the POST method, without refreshing or navigating away from the current page.

Example:

```
var xhttp = new XMLHttpRequest();
xhttp.open("POST", "submitForm.php", true);
xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
xhttp.send("name=John&email=john@example.com");
```

This code sends the user's name and email to a server-side script (submitForm.php) for processing.

3. Displaying Live Updates (Like Scores or Notifications)

AJAX can repeatedly fetch new data at intervals (polling) and update the page, useful for live cricket scores, stock prices, notifications, etc.

Example (auto-refresh every 5 seconds):

```
function fetchUpdates() {  
    var xhttp = new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status == 200) {  
            document.getElementById("liveScore").innerHTML = this.responseText;  
        }  
    };  
    xhttp.open("GET", "score.php", true);  
    xhttp.send();  
}  
  
setInterval(fetchUpdates, 5000); // Fetches new score every 5 seconds
```

This dynamically updates the score section of a webpage every few seconds.

4. Communicating with Server Using GET or POST

Use GET when sending small data in the URL or use POST when sending larger data securely in the body.

GET Example:

```
xhttp.open("GET", "search.php?query=book", true);  
xhttp.send();
```

NOTE: Use GET for simple queries (like search)

POST Example:

```
xhttp.open("POST", "submit.php", true);  
xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");  
xhttp.send("username=admin&password=1234");
```

NOTE: Use POST for sending form data, login credentials, or files.

5. Receiving and Processing Data in XML, JSON, or Plain Text

Fetch different types of data formats from the server and handle them in JavaScript.

Text Example:

```
xhttp.open("GET", "info.txt", true);  
xhttp.send();
```

XML Example:

```
xhttp.onreadystatechange = function() {  
    if (this.readyState == 4 && this.status == 200) {  
        var xmlDoc = this.responseXML;  
        var name =  
xmlDoc.getElementsByTagName("name")[0].childNodes[0].nodeValue;  
        document.getElementById("result").innerHTML = name;  
    }  
};  
xhttp.open("GET", "data.xml", true);  
xhttp.send();
```

JSON Example:

```
xhttp.onreadystatechange = function() {  
    if (this.readyState == 4 && this.status == 200) {  
        var obj = JSON.parse(this.responseText);  
        document.getElementById("user").innerHTML = obj.username;  
    }  
};  
xhttp.open("GET", "user.json", true);  
xhttp.send();
```

Integrating PHP and AJAX

AJAX is frequently used with PHP to create dynamic, server-interactive applications. It can send and receive data to and from PHP scripts without refreshing the page.

When combined with PHP, AJAX can send and receive data from the server dynamically.

The **POST** method is used when we want to send data securely (not visible in URL) and often when inserting/updating records in databases.

In AJAX:

XMLHttpRequest object is created.

`open("POST", "server.php", true)` specifies POST method and target PHP file.

`setRequestHeader("Content-type", "application/x-www-form-urlencoded")` sets header so the server knows how to read the data.

`send("key=value")` sends the data to the server.

On the server side, PHP receives the data using `$_POST[]` and processes it.

To send data from a form to a PHP script, use the POST method in JavaScript:

```
xhttp.open("POST", "server.php", true);  
xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");  
xhttp.send("username=john");
```

The PHP script processes the data:

```
<?php  
echo "Hello, " . $_POST['username'];  
?>
```

This will output a greeting message based on the submitted username.

Example: Sending Data to PHP using POST

HTML:

```
<input type="text" id="username">  
<button onclick="submitData()">Submit</button>  
<div id="response"></div>
```

JavaScript:

```
function submitData() {  
    var user = document.getElementById("username").value;  
    var xhttp = new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status == 200) {  
            document.getElementById("response").innerHTML = this.responseText;  
        }  
    };  
    xhttp.open("POST", "server.php", true);  
    xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");  
    xhttp.send("username=" + user);  
}
```

PHP (server.php):

```
<?php  
$name = $_POST['username'];  
echo "Hello, $name!";  
?>
```

Field Validation Using AJAX and PHP

Field validation ensures user inputs are correct before submission.

With AJAX, we can validate input in real time (e.g., checking if a username already exists).

The GET method is used here since we're just fetching data (not modifying).

Flow:

- User types in a field.
- onkeyup event triggers an AJAX request.
- AJAX sends the data to PHP using GET.
- PHP checks against a database or array of existing values.
- Response (available or taken) is sent back without refreshing the page.

Advantage: Provides immediate feedback to users → improves interactivity.

To validate fields such as checking if a username is already taken, AJAX can be used to call a PHP script that checks the database:

AJAX call:

```
xhttp.open("GET", "check.php?user=john", true);  
xhttp.send();
```

PHP validation script:

```
<?php  
if(username_exists($_GET['user'])) echo 'taken';  
else echo 'available';  
?>
```

The returned result can then be used to show a message to the user about username availability. This real-time feedback makes web forms more interactive and user-friendly.

Example:

HTML:

```
<input type="text" id="user" onkeyup="checkUser()">  
<span id="status"></span>
```

JavaScript:

```
function checkUser() {  
    var user = document.getElementById("user").value;  
    var xhttp = new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status == 200) {  
            document.getElementById("status").innerHTML = this.responseText;  
        }  
    };  
    xhttp.open("GET", "check.php?username=" + user, true);  
    xhttp.send();  
}
```

PHP (check.php):

```
<?php
$users = ["john", "jane", "admin"];
if (in_array($_GET['username'], $users)) {
    echo "Username taken";
} else {
    echo "Available";
}
?>
```

This checks the entered username and returns availability without refreshing the page.